BTLO Challenge Report

Secrets — JWT Analysis & Forgery

Analyst	Abiral Timalsina
Platform	Blue Team Labs Online (BTLO)
Category	Python / Application Security
Difficulty	Easy
Points	10
Tools Used	Kali Linux, base64, Hashcat, OpenSSL, SecLists

1. Scenario

A ticket, presented as a JSON Web Token (JWT), was intercepted showing high-privilege ("admin": true) claims from an unknown source. As the party responsible for the original signing secret, the objective is to identify the token type and structure, recover the signing secret, and forge a new, low-privilege token to prove the vulnerability has been understood and can be remediated.

Original token:

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmbGFnIjoiQlRMe180X0V5ZXN9IiwiaWF0Ijo5MDAwMDAwMCwibmFtZSI6IkdyZWF0RXhwIiwiYWRtaW4iOnRydWV9.jbkZHll_W17BOALT95JQ17glHBj9nY-oWhT1uiahtv8
```

2. Methodology

2.1 Token Decoding

A JWT consists of three Base64URL-encoded sections separated by periods: `Header.Payload.Signature`. The header and payload sections were decoded directly from the command line:

```
echo "<header>" | base64 -d
echo "<payload>" | base64 -d
```

Header: `{"typ":"JWT","alg":"HS256"}`
Payload: `{"flag":"BTL{_4_Eyes}","iat":90000000,"name":"GreatExp","admin":true}`

The payload confirmed the token was signed using HS256 (HMAC-SHA256), a symmetric algorithm requiring a shared secret for both signing and verification — meaning the secret could potentially be recovered through offline cracking.

2.2 Secret Recovery

Standard password wordlists (rockyou.txt) and a dedicated leaked-JWT-secrets wordlist (SecLists' scraped-JWT-secrets.txt) were both tried against the token using John the Ripper and Hashcat's JWT mode (-m 16500), without success — indicating the secret was short and not present in any common wordlist.

A brute-force mask attack was used instead:

```
hashcat -a 3 -m 16500 jwt_hashcat.hash ?a?a?a?a
```

This exhaustively tried all 4-character combinations of printable characters and cracked the secret within seconds: `bT!0`.

2.3 Forging a Low-Privilege Token

The header and a modified payload (with "admin" changed from true to false) were re-encoded manually to preserve the exact key ordering of the original token, since browser-based JWT tools were found to reorder JSON keys and produce a token that would fail an exact-match validation check:

```
HEADER=$(echo -n '{"typ":"JWT","alg":"HS256"}' | base64 -w0 | tr -d '=' | tr '+/' '-_')
PAYLOAD=$(echo -n '{"flag":"BTL{_4_Eyes}","iat":90000000,"name":"GreatExp","admin":false}'
\
| base64 -w0 | tr -d '=' | tr '+/' '-_')
SIGNATURE=$(echo -n "${HEADER}.${PAYLOAD}" | openssl dgst -sha256 -hmac 'bT!0' -binary \
| base64 -w0 | tr -d '=' | tr '+/' '-_')
echo "${HEADER}.${PAYLOAD}.${SIGNATURE}"
```

This produced a byte-exact, validly-signed low-privilege token matching the original header format.

3. Findings

#	Question	Answer
1	Name of the token?	`JWT`
2	Structure of this token?	`Header.Payload.Signature`
3	Hint found from this token?	`_4_Eyes`
4	The Secret?	`bT!0`
5	New verified signature ticket with low privilege?	`eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmbGFnIjoiQlRMe180X0V5ZXN9IiwiaWF0Ijo5MDAwMDAwMCwibmFtZSI6IkdyZWF0RXhwIiwiYWRtaW4iOmZhbHNlfQ.nMXNFvttCvtDcpswOQA8u_LpURwv6ZrCJ-ftIXegtX4`

4. Lessons Learned

• HS256-signed JWTs are only as strong as their secret — short, weak secrets are recoverable via brute-force in seconds, at which point full token forgery becomes trivial.
• Dictionary attacks should be attempted before brute-force, but a short/exhausted keyspace mask attack (e.g. ?a?a?a?a) is often faster than expected for weak secrets and should not be skipped when wordlists fail.
• Automated tools (e.g. browser-based JWT encoders) can silently reorder JSON keys, which may break exact-match signature validation — manually constructing the token guarantees byte-for-byte control over the output.